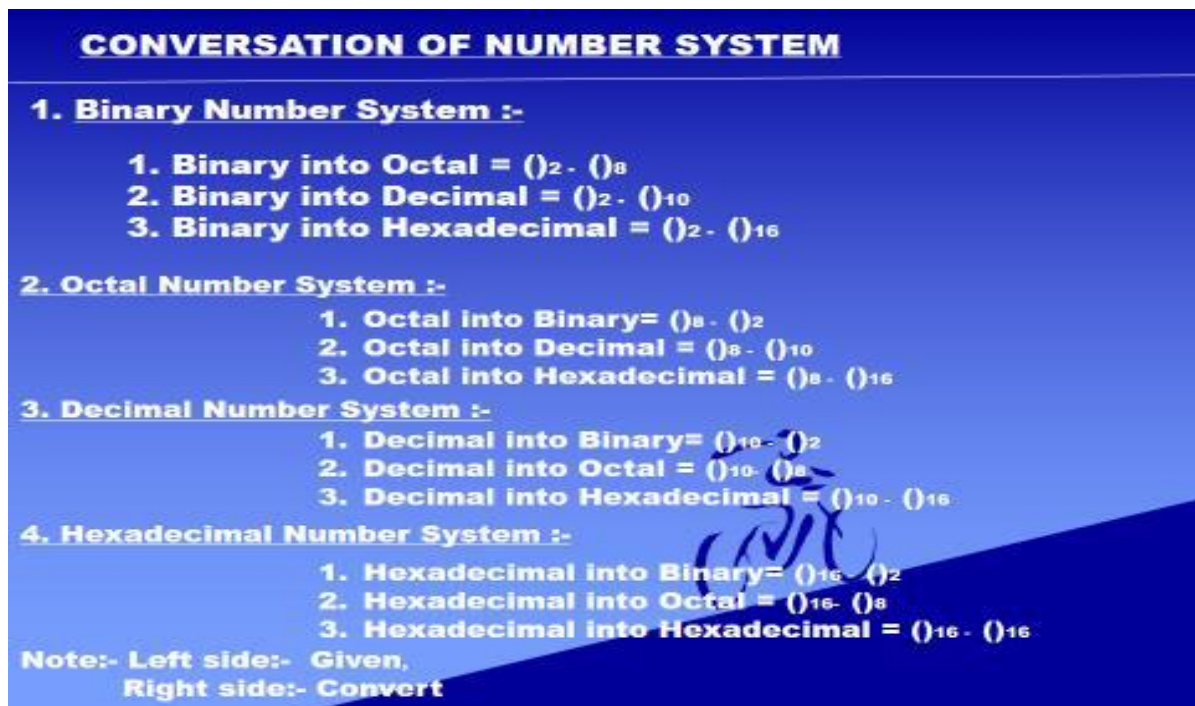
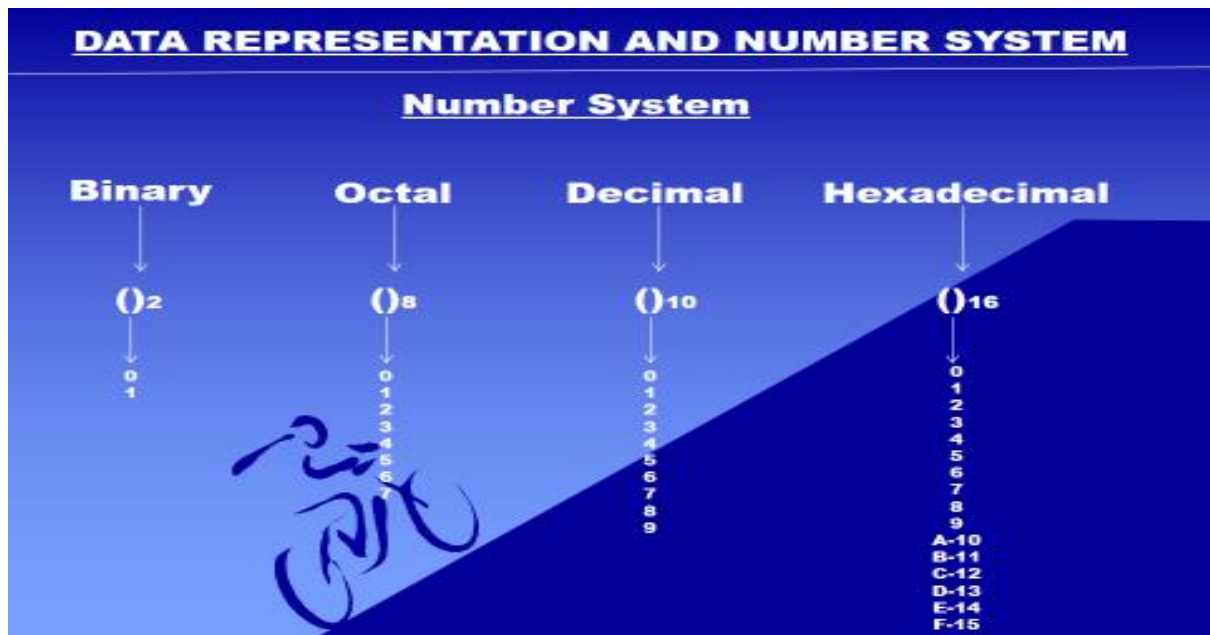


A **number system** is a way of representing numbers using a set of digits or symbols. It defines how numbers are written and interpreted. In computers and digital electronics, number systems are very important because data is stored, processed, and communicated in different number formats.

A number system is mainly characterized by its **base (radix)**, which indicates the total number of unique digits used, including zero.



1. Conversion Binary into Octal - Binary Number is 2
2. Conversion Octal into Binary. Octal Number is 8

1. What should be the power of 2 to get 8?

Ans :- $2^3 = 8$

- Let's see how to make the table.
- Make the table that contains 3 columns and 8 rows.

0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

1. $8/2 = 4$

Write 4 time = 0

Write 4 time = 1

2. $4/2 = 2$

Write 2 time = 0

Write 2 time = 1

3. $2/2 = 1$

Write 1 time = 0

Write 1 time = 1

*Use this table binary to octal & octal to binary.

0 to 1 is used binary.

0 to 7 is used octal.

***Convert:-**

Q-1:- $(761)_8 = ()_2 ?$

Ans - 7 - 1 1 1

6 - 1 1 0

1 - 0 0 1

(111 110 001)

Q-2:- Convert $(71.3)_8 = ()_2 ?$

Ans - 7 - 1 1 1

1 - 0 0 1

.

3 - 0 1 1

(111 001. 011)

Q-3:- Convert $(10011)_2 = ()_8 ?$

value divide by /3

Ans - (010) (011)

2 3

(23)₈

Q-4:- Convert $(10.11)_2 = ()_8 ?$

Ans - (010.110)

(2 . 6)₈

- (3) Conversion Binary into Hexadecimal.
 (4) Conversion Hexadecimal into Binary.

Binary is into = 2
 Hexadecimal is into =16

Q-1. What should be the power of 2 to get 16 ?

$$2^? = 16$$

$$2^4 = 16$$

- Let's see how to make the table.
- Make the table that contains 4 columns and 16 rows.



	0	0	0	0	0
1	0	0	0	0	1
2	0	0	1	0	0
3	0	0	1	0	1
4	0	1	0	0	0
5	0	1	0	0	1
6	0	1	1	0	0
7	0	1	1	0	1
8	1	0	0	0	0
9	1	0	0	0	1
A10	1	0	1	0	0
B11	1	0	1	0	1
C12	1	1	0	0	0
D13	1	1	0	0	1
E14	1	1	1	0	0
F15	1	1	1	1	0

1. $16/2= 8$

Write 8 time = 0

Write 8 time = 1

2. $8/2= 4$

Write 4 time = 0

Write 4 time = 1

3. $4/2= 2$

Write 2 time = 0

Write 2 time = 1

4. $2/2=1$

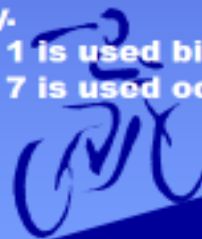
Write 1 time = 0

Write 1 time = 1

*Use this table binary to octal & octal to binary.

0 to 1 is used binary.

0 to 7 is used octal.



Ready for table
 Hexadecimals.

Convert:-*Q-1:- $(61)_{16} = ()_2$?****Ans:- $(0110\ 0001)_2$** **Q-2:- $(8A.D)_{16} = ()_2$?****Ans:- $(10001010.1101)_2$** **Q-3:- $(101101)_2 = ()_{16}$?****Ans:-value divide by/4****0010 1101** **$(2D)_{16}$** **Q-4:- $(11.110)_2 = ()_{16}$?****Ans:-****0011.1100** **$(3.c)_{16}$** **(5) Conversion Octal into Hexadecimal.****Octal—→convert Binary—→ convert—→Hexa-Decimal****(6) Conversion Hexadecimal into Octal.****Hexa-Decimal—→convert Binary—→ convert—→Octal****Q-1:-Convert $(76.1)_8 = ()_{16}$?****Ans:-Convert Octal into Binary** **$(111\ 110.001)_2$** **Convert into Hexa-Decimal**

0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1



(111110.001)₂**11 1110.001****0011 1110.0010****(3 E.2)₁₆**

0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
A10	1	0	1	0
B11	1	0	1	1
C12	1	1	0	0
D13	1	1	0	1
E14	1	1	1	0
F15	1	1	1	1

Binary Arithmetic

Binary arithmetic involves performing arithmetic operations (addition, subtraction, multiplication, and division) using binary numbers, which are numbers expressed in the base-2 numeral system with only two digits: 0 and 1.

Binary Addition

As explained earlier, binary addition is done following the same rules as decimal addition. When adding two binary numbers, you add each pair of corresponding digits from right to left, considering any carry to the next position if the sum is 2 (10 in binary).

$$0+0=0$$

$$0+1=1$$

$$1+0=1$$

$$1+1=0 \text{ WITH } 1 \text{ CARRY}$$

Decimal Number System.



1. $1+1=0$ with 1 carry.

$$\begin{array}{r} 1 \\ + 1 \\ \hline 2 = 10 \text{ (carry next column)} \end{array}$$

2. $1+1+1=1$ with 1 carry.

$$\begin{array}{r} 1 \\ + 1 \\ + 1 \\ \hline 3 = 11 \text{ (carry next column)} \end{array}$$

ex-1:-

$$\begin{array}{r} 11 \\ + 11 \\ \hline 1100 \end{array}$$



Binary Subtraction

Binary subtraction is a fundamental operation in binary arithmetic, just like in decimal arithmetic. It involves subtracting one binary number from another. Here's how binary subtraction works:

Let's say you want to subtract a binary number (the subtrahend) from another binary number (the minuend). You start from the rightmost (least significant) bits of both numbers and move towards the leftmost (most significant) bits, borrowing when necessary, just like you do in decimal subtraction.

Here's a step-by-step process for binary subtraction:

Start from the rightmost bits (least significant bits) of both numbers.

Subtract the bit from the subtrahend from the corresponding bit in the minuend.

If the minuend bit is smaller than the subtrahend bit, you need to borrow. Borrowing in binary means borrowing 1 from the next higher bit.

Repeat the process for each bit, moving from right to left. Write down the results, which will be the bits of the difference (result of subtraction).



Starting from the rightmost bit:

$$1 - 1 = 0$$

$$0 - 1 = 1 \text{ (borrow 1 from the next higher bit)}$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

example subtract:--

	(2)	(2)	2	2
	0(10)	0(10)	0 1 1 10	0 110
1. 111	2. 110 1 0	3. 1 1 1 0 1	4. 1 1 0 0 0	5. 1 0 0 0 1 0 0
- 01	- 0 1	- 1 1	- 1	- 1
-----	-----	-----	-----	-----
110	110 0 1	1 1 0 1 0	1 0 1 1 1	1 0 0 0 0 1 1

1'st Complements:-

* Binary Number System is one the type of most popular Number Representation techniques that used in digital systems. In the Binary System, there are only two symbols or possible digit values, i.e., 0 (off) and 1 (on). Represented by any device that only 2 operating states or possible conditions.

* Generally, there are two types of complement of Binary number: 1's complement and 2's complement. To get 1's complement of a binary number, simply invert the given number. For example, 1's complement of binary number 110010 is 001101. To get 2's complement of binary number is 1's complement of given number plus 1 to the least significant bit (LSB). For example 2's complement of binary number 10010 is (01101) + 1 = 01110.

Example-1: Find 1's complement of binary number 10101110.

Simply invert each bit of given binary number, so 1's complement of given number will be 01010001.

Example-2: Find 1's complement of binary number 10001.001.

Simply invert each bit of given binary number, so 1's complement of given number will be 01110.110.

Example-3: Find 1's complement of each 3 bit binary number.

Simply invert each bit of given binary number, so 1's complement of each 3 bit binary number will be,

<u>Binary number</u>	<u>1's complement</u>
000	111
001	110
010	101
011	100
100	011
101	010
110	001
111	000

Uses of 1's Complement Binary Numbers:

There are various uses of 1's complement of Binary numbers, mainly in signed Binary number representation and various arithmetic operations for Binary numbers, e.g., additions, subtractions, etc.

1's Complement in Signed Binary number Representation:

1's complement binary numbers are very useful in Signed number representation. Positive numbers are simply represented as Binary number. There is nothing to do for positive binary number. But in case of negative binary number representation, we represent in 1's complement. If the number is negative then it is represented using 1's complement. First represent the number with positive sign and then take 1's complement of that number.

Example: Let we are using 5 bits register. The representation of -5 and +5 will be as follows:

+5 is represented as it is represented in sign magnitude method. -5 is represented using the following steps:

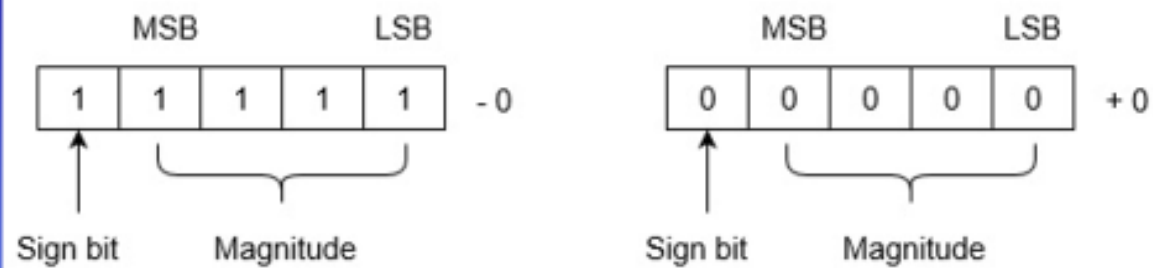
(i) $+5 = 0\ 0101$

(ii) Take 1's complement of $0\ 0101$ and that is $1\ 1010$. MSB is 1 which indicates that number is negative.

MSB is always 1 in case of negative numbers.

Range of Numbers: For k bits register, positive largest number that can be stored is $(2^{(k-1)} - 1)$ and negative lowest number that can be stored is $-(2^{(k-1)} - 1)$.

Note that drawback of this system is that 0 has two different representation one is -0 (e.g., $1\ 1111$ in five bit register) and second is +0 (e.g., $0\ 0000$ in five bit register).



complement binary numbers.

Subtractions by 1's Complement:

The algorithm to subtract two binary number using 1's complement is explained as following below:

Take 1's complement of the subtrahend

Add with minuend

If the result of above addition has carry bit 1, then add it to the least significant bit (LSB) of given result

If there is no carry bit 1, then take 1's complement of the result which will be negative

Note that subtrahend is number that to be subtracted from the another number, i.e., minuend.

Example (Case-1: When Carry bit 1): Evaluate 10101 - 00101

According to above algorithm, take 1's complement of subtrahend 00101, which will be 11010, then add both of these. So, $10101 + 11010 = 1\ 01111$. Since, there is carry bit 1, so add this to the LSB of given result, i.e., $01111 + 1 = 10000$ which is the answer.

Example (Case-2: When no Carry bit): Evaluate 11110 with 1110

According to above algorithm, take 1's complement of subtrahend 11110, which will be 00011. Then add both of these, So, $11001 + 00011 = 11100$. Since there is no carry bit 1, so take 1's complement of above result, which will be 00011, and this is negative number, i.e., 00011, which is the answer.

Similarly, you can subtract two mixed (with fractional part) binary numbers. Note that you always add Carry bit the the least significant bit (LSB) of the result, whenever you get carry bit 1. LSB of fractional binary number is last (rightmost) bit of mixed or fractional binary numbers.

Additions by 1's Complement:

There are different scenarios for addition of two binary numbers using 1's complement. These are explained as follows below.

Case-1: Addition of positive and negative number when positive number has greater magnitude:

When positive number has greater magnitude, then take simply 1's complement of negative number and the end-around carry of the sum is added to the least significant bit (LSB).

Example: Add 1110 and -1101.

So, take 1's complement of 1101, which will be 0010, then add with given number. So, $1110 + 0010 = 10000$, then add this carry bit to the LSB, $0000 + 1 = 0001$, which is the answer.

Note that if the register size is big then fill the same value of MSB to preserve sign magnitude for inputs and output.

Case-2: Addition of positive and negative number when negative number has greater magnitude:

When the negative number has greater magnitude, then take 1's complement of negative number and add with given positive number. Since there will not be any end-around carry bit, so take 1's complement of the result and this result will be negative.

Example: Add 1010 and -1100 in five-bit registers.

Note that there are five-bit registers, so these new numbers will be 01010 and -01100. Now take 1's complement of 01100 which will be 10011 and add $01010 + 10011 = 11101$. Then take 1's complement of this result, which will be 00010 and this will be negative number, i.e., -00010, which is the answer.

Case-3: Addition of two negative numbers:

You need to take 1's complement for both numbers, then add these 1's complement of numbers. Since there will always be end-around carry bit, so add this again to the MSB of result. Now, take 1's complement also of previous result, so this will be negative number.

Alternatively, you can add both negative number directly, and get this result which will be negative only.

Example: add -1010 and -0101 in five bit-register.

These five bit numbers are -01010 and -00101. Add complements of these numbers, $10101 + 11010 = 1\ 01111$. Since, there is a carry bit 1, so add this to the LSB of result, i.e., $01111 + 1 = 10000$. Now take the 1's complement of this result, which will be 01111 and this number is negative, i.e., -01111, which is answer.

Note that end-around-carry-bit addition occurs only in 1's complement arithmetic operations but not in 2's complement arithmetic operations.



2's complement

This is one of the methods of representing signed integers in the computer. In this method, the most significant digit (MSD) takes on extra meaning.

If the MSD is a 0, we can evaluate the number just as we would any normal unsigned integer.

If the MSD is a 1, this indicates that the number is negative.

The other bits indicate the magnitude (absolute value) of the number.

If the number is negative, the other bits signify the 2's complement of the magnitude of the number.

Thus a positive number has the same representation in SM, 1's complement, and 2's complement notations. Only negative numbers are represented differently in these notations. Some signed decimal numbers and their equivalent in 2's complement notation is shown below, assuming a word size of 4 bits.

Signed decimal	2's complement
----------------	----------------

+6	0110
----	------

-6	1010
----	------

+0	0000
----	------

+7	0111
----	------

-7	1001
----	------

Notice that there is a single notation for 0, immaterial of whether it is +0 or -0. One may feel that 0 000 is +0 only, as the MSB, in this case, is a 0. But then, the notation for -0 should be the 2's complement of 0 000, which is $1111 + 1 = 0\ 000$ ignoring the carry.

Thus, in 2's complement, notation an extra negative number can be represented compared with Signed Magnitude or 1's complement notation. This is because, in 2's complement notation, there is only a single notation for zero, whereas in SM and 1's complement notations there are two notations for 0.

Exam1- $(1100)_2 - (1010)_2$

1'st complement of 1010 is 0101

2'st

1
1100
+0101

10001

1
0001
+ 1

Result 0010

Exam 2 - $(1010)_2 - (1100)_2$

1'st complement of 1100 is 0011.

2'st

1
1010
+ 0011

1101

flip the number 0010

Multiplication :- Binary multiplication is the process of multiplying two binary numbers together. Binary numbers are numbers expressed in the base-2 numeral system, which only uses the digits 0 and 1. The multiplication process is similar to decimal multiplication, but simpler because there are only two possible digits.

Rules:-

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

Ex-1.
$$\begin{array}{r} 11010 \\ \times 011 \\ \hline \text{carry } (1)11010 \\ (1)11010 \\ 00000 \\ \hline 1001110 \end{array}$$



❖ Division Method:

Binary division is the process of dividing **binary numbers (0 and 1)**, just like decimal division, but using **base-2 rules**.

Rules for binary Division		
Number		Division
A	B	A / B
0	1	0
1	1	1
0	0	
1	0	NOT POSSIBLE

EXAMPLE:

1. Divide 11011011 By 101.

$$\begin{array}{r} 101011 \\ 101 \overline{) 11011011} \\ \underline{101} \\ 00111 \\ \underline{101} \\ 01001 \\ \underline{101} \\ 1001 \\ \underline{101} \\ 100 \end{array}$$

Quotient = 101011
And Remainder = 100

Prove: $219 / 5 = 43$
Remainder = 4

2. Divide 1111111 By 111.

$$\begin{array}{r} 100100 \\ 111 \overline{) 1111111} \\ \underline{111} \\ 000111 \\ \underline{111} \\ 000111 \\ \underline{111} \\ 00011 \end{array}$$

Quotient = 100100
And Remainder = 11

Prove: $255 / 7 = 36$
Remainder = 3